

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***ASSIGNMENT COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures & Patterns
Assignment number and title: 3 – Amortized Analysis & Abstract Data Types
Due date: Sunday, May 18, 2025, 23:59
Lecturer: Dr. Markus Lumpe

Your name: Chrishelle Anthoneyz **Your student ID:** 104067504

Marker's comments:

Problem	Marks	Obtained
1	34	
2	112	
Total	146	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____

```
// COS30008 - Problem Set 3, 2025
```

```
#pragma once
```

```
#include <ostream>
```

```
template<typename K, typename V>
```

```
class SortablePair {
```

```
public:
```

```
    // construct from key & value
```

```
    SortablePair(const K &aFirst = K{}, const V &
aSecond = V{}) noexcept
        : fFirst(aFirst), fSecond(aSecond) {
    }

```

```
    // Access the key
```

```
    const K &first() const noexcept { return fFirst; }
```

```
    // Access the value
```

```
    const V &second() const noexcept { return fSecond
; }
```

```
    // True if both key and value match
```

```
    bool operator==(const SortablePair &aOther) const
noexcept {
        return fFirst == aOther.fFirst
            && fSecond == aOther.fSecond;
    }

```

```
    // Define "less" by higher key first
```

```
    bool operator<(const SortablePair &aOther) const
noexcept {
        return fFirst > aOther.fFirst;
    }

```

```
    // Stream output as (key,value)
```

```
    friend std::ostream &operator<<(std::ostream &os,
const SortablePair &p) {
        return os << '(' << p.fFirst << ',' << p.
fSecond << ')';
    }

```

```
private:  
    K fFirst; // priority  
    V fSecond; // associated value  
};
```

```
// COS30008 - Problem Set 3, 2025
```

```
#pragma once
```

```
#include "SortablePair.h"
```

```
#include <optional>
```

```
#include <cassert>
```

```
#include <algorithm>
```

```
#include <new>
```

```
template<typename T, typename P>
```

```
class PriorityQueue {
```

```
public:
```

```
    using value_type = SortablePair<P, T>;
```

```
    // Start empty with capacity 1
```

```
    PriorityQueue() noexcept
```

```
        : fHead(0), fTail(0), fCapacity(1) {
```

```
            fElements = static_cast<value_type*>(
                ::operator new(sizeof(value_type) *
fCapacity)
```

```
            );
```

```
    }
```

```
    // Destroy all elements and free buffer
```

```
    ~PriorityQueue() noexcept {
```

```
        // call destructors for live elements
```

```
        for (size_t i = fHead; i < fTail; ++i)
```

```
            fElements[i].~value_type();
```

```
        ::operator delete(fElements);
```

```
    }
```

```
    PriorityQueue(const PriorityQueue &) = delete;
```

```
    PriorityQueue &operator=(const PriorityQueue &) =
delete;
```

```
    // Number of items in queue
```

```
    size_t count() const noexcept { return fTail -
fHead; }
```

```

    // Current buffer capacity
    size_t capacity() const noexcept { return fCapacity
; }

    // highest-priority value, or nullopt if empty
    std::optional<T> top() const noexcept {
        if (count() == 0) return std::nullopt;
        return fElements[fHead].second();
    }

    // insert aValue with priority aPriority
    void push(const T &aValue, const P &aPriority)
noexcept {
        ensure_capacity();
        // placement-new at tail
        new(&fElements[fTail++]) value_type(aPriority,
aValue);
        // reorder [fHead, fTail)
        sort();
    }

    // remove highest-priority element
    void pop() noexcept {
        assert(count() > 0);
        // destroy head element
        fElements[fHead++].~value_type();
        adjust_capacity();
    }

private:
    value_type *fElements; // raw storage array
    size_t fHead; // index of first valid element
    size_t fTail; // one-past-last valid element
    size_t fCapacity; // allocated slots

    // sort active window by priority
    void sort() noexcept {
        std::sort(&fElements[fHead], &fElements[fTail
]);
    }

```

```

    void resize(size_t aCapacity) noexcept {
        // allocate new buffer
        value_type *newBuf = static_cast<value_type *>(
            ::operator new(sizeof(value_type) *
aCapacity)
        );
        // move-construct live elements
        size_t c = count();
        for (size_t i = 0; i < c; ++i)
            new(&newBuf[i]) value_type(fElements[fHead
+ i]);

        // destroy old elements
        for (size_t i = fHead; i < fTail; ++i)
            fElements[i].~value_type();
        ::operator delete(fElements);

        // update indices and capacity
        fElements = newBuf;
        fCapacity = aCapacity;
        fHead = 0;
        fTail = c;
    }

    // Double capacity if tail == capacity.
    void ensure_capacity() noexcept {
        if (fTail == fCapacity)
            resize(fCapacity * 2);
    }

    // Halve capacity if count ≤ capacity/4.
    void adjust_capacity() noexcept {
        size_t c = count();
        if (fCapacity > 1 && c <= fCapacity / 4)
            resize(fCapacity / 2);
    }
};

```